

CS201 – Data Structures II

Lecture 02

Dr. Muhammad Mobeen Movania

Outline

- Mathematical Preliminaries
- Asymptotic notation
- Inclusion relations
- Model of computation
- Running times

Mathematical Preliminaries

- Exponentials and Logarithms

$$\log_b k = x$$

$$b^x = k$$

- We usually consider base 2 logs
- Log base conversion formula

$$\log_b k = \frac{\log_a k}{\log_a b} .$$

- When base $b = \exp$, we have natural log ($\ln$)

Mathematical Preliminaries

- Factorials

$$n! = 1 * 2 * 3 * \cdots * n$$

- Counts the number of distinct permutations of n distinct elements

- Binomial coefficients

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

- Gives the number of ways of choosing k distinct integers from the set $\{1, \dots, n\}$

Asymptotic Notation

- Its difficult to compute the running time of an algorithm or a data structure
- We therefore use big-Oh notation to analyze data structures and algorithms
- Its is defined as the following

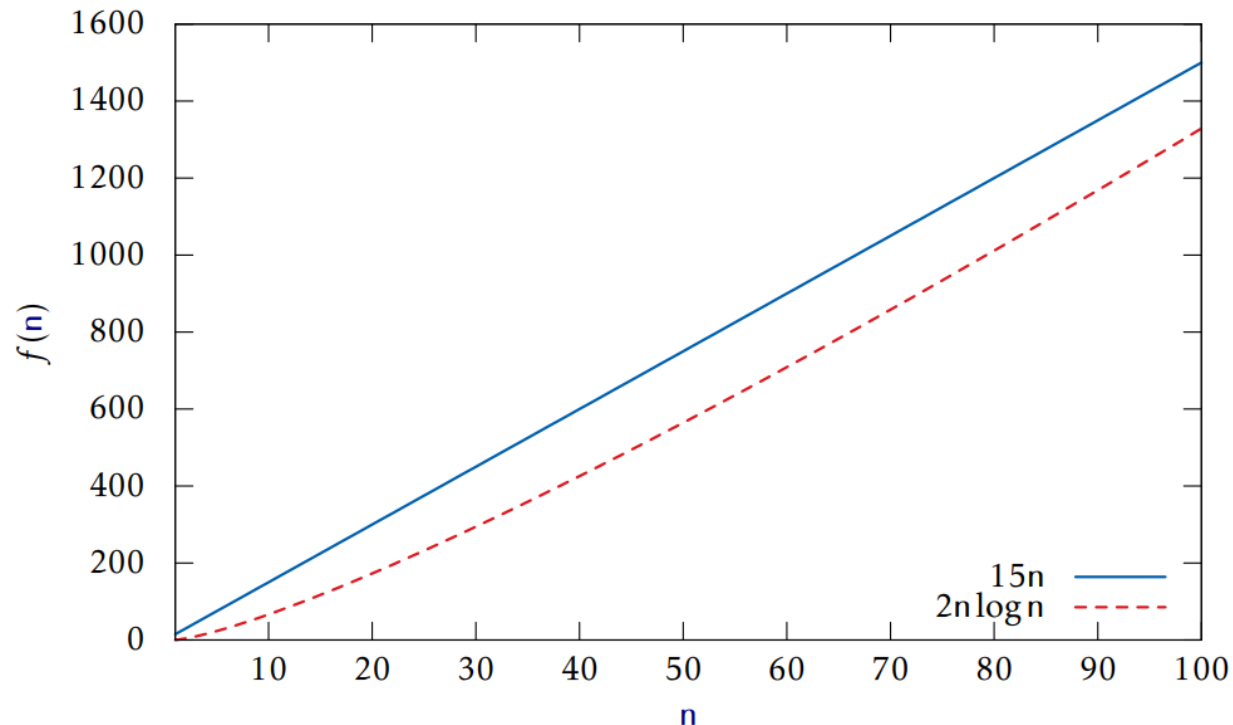
$$O(f(n)) = \left\{ \begin{array}{l} g(n) : \text{there exists } c > 0, \text{ and } n_0 \text{ such that} \\ g(n) \leq c \cdot f(n) \text{ for all } n \geq n_0 \end{array} \right\}$$

- this set consists of the functions $g(n)$ where **$c \cdot f(n)$ starts to dominate $g(n)$ when n is sufficiently large.**

Inclusion relations

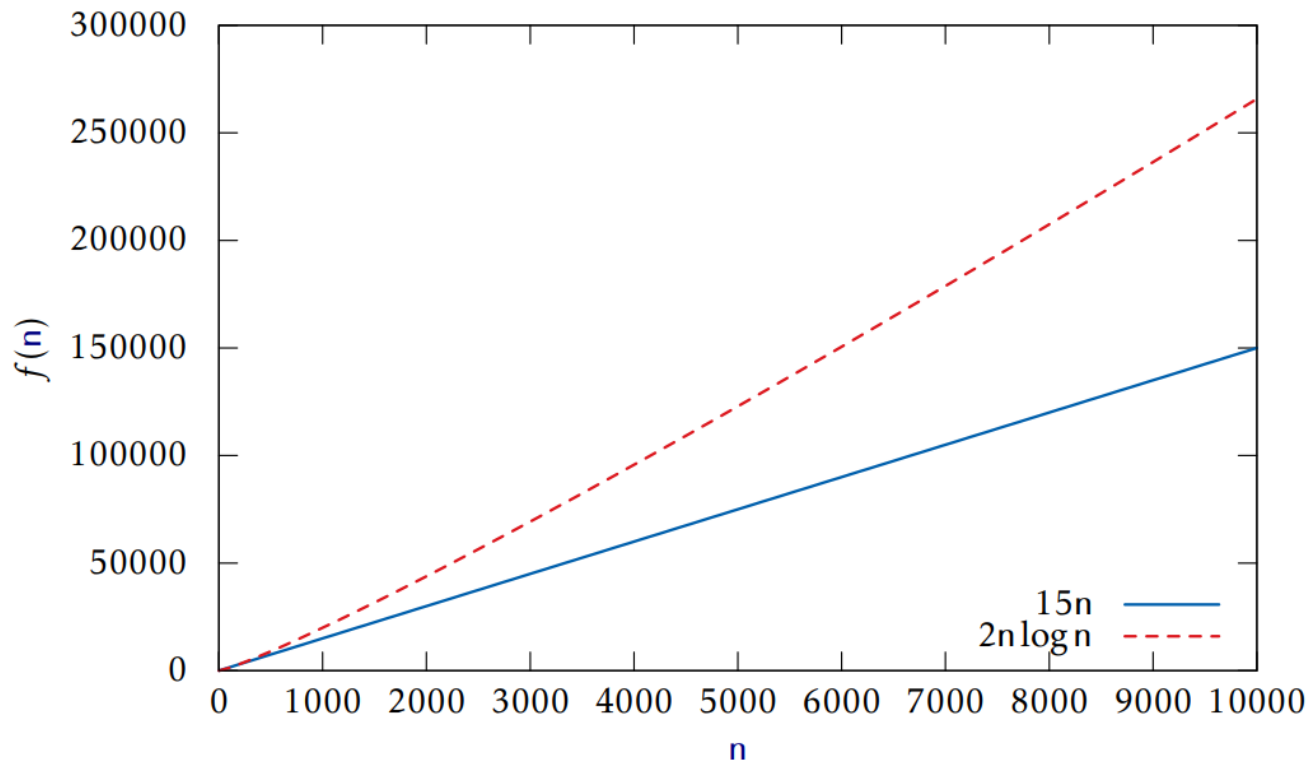
$$O(n) \subset O(n \log n) \subset O(n^{1+b}) \subset O(nc^n)$$

- Comparing two functions, $15n$ and $2n \log(n)$
 - For smaller n , $15n$ dominates but ...



Inclusion relations

- Comparing two functions, $15n$ and $2n\log(n)$
 - For larger values of n , $2n\log(n)$ dominates



Model of Computation

- w -bit word-RAM (Random Access Machine) model (with $w \geq \log(n)$)
- In the word-RAM model, basic operations on words take constant time.
 - includes arithmetic operations (+, −, *, /, %), comparisons (<, =, ≤, ≥), and bitwise boolean operations (bitwise-AND, OR, and exclusive-OR).
- Example
 - Allocating a block of memory of size k takes $O(k)$ time and returns a reference (a pointer) to the newly-allocated memory block.

Running Times

- Three different running times guarantees
 - Worst-case running time
 - Amortized running time
 - Expected running time
- Worst-case running times
 - Strongest kind of running time guarantee
 - If a data structure operation has a worst-case running time of $f(n)$, then one of these operations never takes longer than $f(n)$ time.

Running Times

- Amortized running time
 - Tells that the cost of a typical operation is at most $f(n)$
 - if a data structure has an amortized running time of $f(n)$, then a sequence of m operations takes at most $m \cdot f(n)$ time
 - Some operations may take more than $f(n)$ but the average, over the entire sequence of operations, is at most $f(n)$.

Running Times

- Expected running time
 - If the expected running time of a data structure operation is $f(n)$, this means that the actual running time is a random variable and the expected value of this random variable is at most $f(n)$
- Why should we learn all these?
 - It is often possible to get a **lower expected or amortized running time** than a worst-case running time for a data structure operation

Book Readings

- ODS: Chapter 1 pages: 10-21

Next Lecture

- Lecture 03